1768. Merge Strings alternately

word1 = "abc" , word2 = "pqrstu"

o/p apbq crstu → merged string

word1 = "abcd" , word2 = "pq"

o/p = apbqcd

eg.:

i = 0

abc
↑

pqr
↑

ans = " "

ans = ap

i = 1

abc
↑

pqr
↑

ans = apbq

i = 2

abc
↑

pqr
↑

ans = apbqcr → o/p

```
{
 int n1 = word1.size();
 int n2 = word2.size();
 int i=0;
 string ans = "";
while( i < n1 && i < n2){
     ans = ans + word1[i] + word2[i];
     i++;
 }
 while (i < n1) {
      ans = ans + word1[i];
      i++;
 while( i < n2){
      ans = ans + word2[i];
        i++;
    }
   return ans;
 }
};
```

eg

size=2 size=4

ab pqrs

$i=0$ $n_1 < n_2$

ans = ap

ab pqrs

$i=1$

ans = apbq

ab pqrs

$i=3$

ans = apbqrs

$i=3$, n_1 size = 2

$i < n_1 \Rightarrow 3 < 2$

False

$i=3$, n_2 size = 4

$i < n_2 \Rightarrow 3 < 4$ True

word2 append

1754. largest merge of 2 Strings

word1: caba a word2: bcaaa

i=0 j=0

c > b ✓

merge = [c]

i++

abaa bcaaa

a > b ✗

merge = [c, b] → list

j++

abaa caaa

a > c ✗

merge = [c, b, c]

j++

abaa aaa

a == a

a > a ✓

merge [c, b, c, a]

i++

baa aaa

b > a ✓

merge [c, b, c, a, b]

i++

aa aaa

a > a

Time = $O(n^2)$

space = $O(n)$

Code

```
class Solution:
    def largestMerge( self, word1: str,
        word2: str) → str:
        i=0
        j=0
        merge=[]
        while i< len(word1) and j< len(word2):
            if word1[i:] > word2[j:]:
                merge.append(word1[i])
                i+=1
            else:
                merge.append(word2[j])
                j+=1
        merge.append(word1[i:])
        merge.append(word2[j:])
        return "".join(merge)
```

443. String Compression

i/p => chars = {'a', 'a', 'b', 'b', 'c', 'c', 'c'}

o/p => 6

{'a', '2', 'b', '2', 'c', '3', 'c'}

length = 6

not matter

	0	1	2	3	4	5	6
arr =	a	a	b	b	c	c	c
	~~i~~	~~i~~	i				

curr_char = 'a'

count = 2

a	a	b	b	c	c	c

index i

curr_char	count	b	b	c	c	c

index i

a	2	b	b	c	c	c

i

	0	1	2	3	4	5	6
	a	a	b	b	c	c	c
	~~i~~	~~i~~	i				

curr_char = 'b'

count = 2

i++

a	2	b	b	c	c	c

index ~~i~~ i

index++

'a'	'2'	'b'	'2'	'c'	'c'	'c'

index i

arr = | a | 2 | b | 2 | c | c | c |

index ~~i~~ ~~i~~ ~~i~~ i

=> curr_char = c

count = 3

assign count

assign

Now assign => | a | 2 | b | 2 | c | c | c |

index i

index++

a	2	b	2	c	3	c

index i

0	1	2	3	4	5	6
'a'	'2'	'b'	'2'	'c'	'3'	'c'

index

i

i is out of loop so stop.

index value = 6 Ans

eg

0	1	2	3	4	5	6	7	8	9	10	11
a	a	a	a	a	a	a	a	a	a	b	b

output = 5

index

curr_char = 'a'

count = 10

Now assign as a ≠ b

→ 10 → '10'

count convert into string

0	1	2	3	3	4	5	6	7	8	9	10	11
a	1	0	a	a	a	a	a	a	a	a	b	b

index index index

i

⇒

0	1	2	3	4	5	6	7	8	9	10	11
a	1	0	a	a	a	a	a	a	a	b	b

index

curr_char = 'b'

count = 2

Now assign ⇒

off

0	1	2	3	4	5	6	7	8	9	10	11
a	1	0	b	2	a	a	a	a	a	b	b

index index

i

Code

eg.: {a, b}

o/p ⇒ {a b} = 2

not ⇒ {a 1 b 1} ×

count

write when count > 1

```
class Solution {
public:
int compress (vector <char> & chars) {
    int n = chars.size();
    int index=0;  int i=0;

    while (i<n) {
        char curr_char = chars[i];
        int count =0;
        // find count of duplicates
        while (i<n && chars[i]== curr_char) {
            count++;    i++;
        }
        // No do the assign work
        chars[index] = curr_char;
        index++;
        if (count > 1) {
            string count_str = to_string(count);
            for(char &ch : count_str) {
                chars[index] = ch;
```

```
            index++;
        }
    }
    return index;
}
```